2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI) | 979-8-3315-2871-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/IDICAIEI61867.2024.10842757

Adaptive Conversation Recommendation Systems: Leveraging Large Language Models and Knowledge Graphs

Sagar Mankari
AI Lead, Manager
PwC
Denver, CO, United States
sagar.mankari@pwc.com

Abhishek Sanghavi
AI Strategy Lead, Director
PwC
Tampa, FL, United States
abhishek.sanghavi@pwc.com

***Abstract*—This paper introduces a novel conversation recommendation system that leverages large language models (LLMs) and knowledge graphs (KGs) to enhance recommendation accuracy and adaptability. Our system dynamically analyzes user queries using LLMs, decomposing them into meaningful graph nodes within a reference KG like YAGO. These nodes then form the basis of personalized user-centric KGs, representing individual preferences. Node weights are adjusted based on ongoing conversations, capturing evolving user intent. To ensure relevance, decay rate factor is implemented to reduce weights on less frequently utilized nodes. This dual mechanism allows our system to provide increasingly relevant recommendations while adapting to changing user interests. Experimental results demonstrate the effectiveness of our approach, showcasing improved recommendation performance compared to traditional methods.**

***Index Terms*—Conversation Recommendation Systems, Large Language Models, Knowledge Graphs, Adaptive Node Weighting, Personalized Recommendations**

I. Introduction

Conversation recommendation systems play a critical role in providing personalized and relevant recommendations. To address limitations in capturing the nuances of user intent and preferences, we propose a novel approach combining the power of a custom-built Large Language Model (LLM) agent based on the GPT-4o model and a custom user-centric knowledge graph (KGs), drawing inspiration from recent advancements in knowledge graph-based recommendation systems [1]–[4] and hybrid approaches that integrate KGs with deep learning models [5], [6], and the growing capabilities of LLMs in understanding and responding to complex user queries [7], [8]. Our system dynamically analyzes user queries using LLMs, breaking them down into meaningful graph nodes within a reference KG like YAGO. These nodes form the basis of personalized user-centric KGs, capturing individual preferences. By adjusting node weights based on ongoing conversations, and capture user interest in real-time. Additionally, a decay rate factor is implemented to reduce weights on less frequently utilized nodes, ensuring the system remains adaptive to changing user interests. User-centric KGs are built using neo4j graph database.

This paper presents the detailed design and implementation of a novel system, along with an extensive experimental evaluation to assess its effectiveness relative to traditional conversation recommendation methods. The results demonstrate significant improvements achieved by this approach, underscoring its potential to enhance user experience across various applications.

Key contributions:

- A novel conversation recommendation framework leveraging LLMs and KGs to build personalized user-centric KGs
- Dynamic decomposition of user queries into meaningful graph nodes.
- Adaptive node weighting based on ongoing conversations to capture user intent.
- Implementation of a decay rate factor to reduce weights on less frequently utilized nodes.
- Experimental validation demonstrating the effectiveness of the proposed system.

We believe our system represents a significant step towards more intelligent and personalized conversation recommendation systems. By effectively leveraging LLMs, KGs, and adaptive weighting mechanisms, we can provide users with recommendations that are both accurate and adaptable to their evolving needs and preferences.

II. Related Work

The field of conversational recommendation systems has witnessed significant advancements, driven by the growing need for personalized and interactive recommendation experiences. This section delves into the related work, highlighting key research areas that have contributed to the development of our proposed system.

A. Knowledge Graph-based Recommendation

Knowledge graphs (KGs) have emerged as a powerful tool for enhancing recommendation systems by providing a structured representation of entities and their relationships [5], [9]. Leveraging KGs allows for a deeper understanding

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on March 15,2026 at 05:25:47 UTC from IEEE Xplore. Restrictions apply.

of user preferences and item characteristics, leading to more accurate and context-aware recommendations. Research has explored various techniques for incorporating KGs into recommendation models, including embedding methods, path-based reasoning, and graph neural networks [10], [11]. These approaches aim to effectively utilize the rich semantic information encoded within KGs to improve recommendation quality.

B. Deep Learning and Recommendation

The advent of deep learning has revolutionized recommendation systems, enabling the development of sophisticated models capable of capturing complex user-item interactions. Sequential recommendation models based on recurrent neural networks (RNNs) and attention mechanisms [12], [13] have shown promising results in capturing user behavior patterns and predicting future preferences. Furthermore, the introduction of Transformer-based architectures like BERT4Rec has significantly improved the performance of sequential recommendation tasks.

C. Hybrid Approaches

Combining KGs with deep learning models has gained considerable attention in recent years [14], [15]. Hybrid approaches leverage the strengths of both paradigms, utilizing KGs to provide contextual information and deep learning models to learn intricate patterns from user interactions. These approaches have demonstrated the potential to enhance personalized recommendation systems by dynamically adapting to evolving user preferences and providing more explainable recommendations.

D. Adaptive Weighting and Temporal Dynamics

The incorporation of adaptive weighting mechanisms in recommendation systems has shown promising results in capturing dynamic user interests [16], [17]. By dynamically adjusting the weights of nodes within a KG based on user interactions, the system can prioritize relevant information and adapt to changing preferences. Furthermore, incorporating temporal awareness into recommendation systems allows for capturing the recency of user interactions and providing more timely recommendations [18].

E. Large Language Models (LLMs) in Recommendation

Recent advancements in LLMs, such as GPT-3 and its successors, have opened new avenues for enhancing conversational recommendation systems. LLMs can be used to analyze user queries, understand natural language nuances, and generate more personalized and engaging recommendations. Integrating LLMs with KGs offers the potential to combine the power of language understanding with structured knowledge representation, leading to more intelligent and adaptive recommendation systems.

Our proposed system builds upon these foundations, integrating LLMs and KGs with adaptive weighting mechanisms to create a user-centric and dynamically adaptive conversational recommendation system. By leveraging the strengths of these approaches, we aim to provide more accurate, personalized, and engaging recommendations that cater to the evolving needs and preferences of users.

III. Methodology

Our proposed methodology for enhancing conversation recommendation systems using LLMs and KGs involves several key steps:

A. YAGO-based Entity and Class Identification

Our custom-built LLM agent, utilizing the GPT-4o model, plays a crucial role in identifying the entities and relevant classes from the user's query within the YAGO knowledge graph. This process involves several steps:

- **Query Reception:** The LLM agent receives the user's query, which may contain one or more entities of interest (entities of interest in this case would mostly be the subject of users query).
- **Entity of Interest Identification:** The LLM agent analyzes the query to identify the main entity of interest or multiple entities of interests mentioned. If there are multiple entities, the LLM agent identifies them all.
- **Relevant Class Retrieval:** The LLM agent generates a SPARQL query to retrieve all classes from the YAGO knowledge graph. Then the LLM agent iteratively tries to identify classes which are relevant to the identified entities of interest.
- **Class Exploration:** The LLM agent then explores the retrieved classes, potentially identifying more specific subclasses that are directly related to the identified entities. This step allows for a more granular understanding of the query's context.

B. User-Centric Knowledge Graph Construction

To create a user-centric knowledge graph (KG) that reflects the user's interests and query context, the LLM agent leverages identified entities and relevant classes, constructing multi-relational graphs in the neo4j graph database. This process aims to provide a comprehensive representation of the user's information needs and the context surrounding their query:

- **Node Creation:** For each identified entity of interest and relevant class equivalent in YAGO, the LLM agent creates corresponding class nodes in the user's KG if they don't already exist. If there are relevant subclasses identified, they would be marked as children of class nodes created in the user's KG. Once we have relevant class and subclass representation of the entity of interest, we would go ahead and assign a child node of relevant class / subclass to the entity of interest.
- **Edge Creation:** The LLM agent creates edges between the nodes to represent the relationships between the entities and classes, such as instance-of relationships or other relevant connections.
- **Weight Adjustment:** If an entity of interest already exists in the KG, the LLM agent adjusts its edge weight to its class / subclass, based on factors like the frequency of the

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on March 15,2026 at 05:25:47 UTC from IEEE Xplore. Restrictions apply.

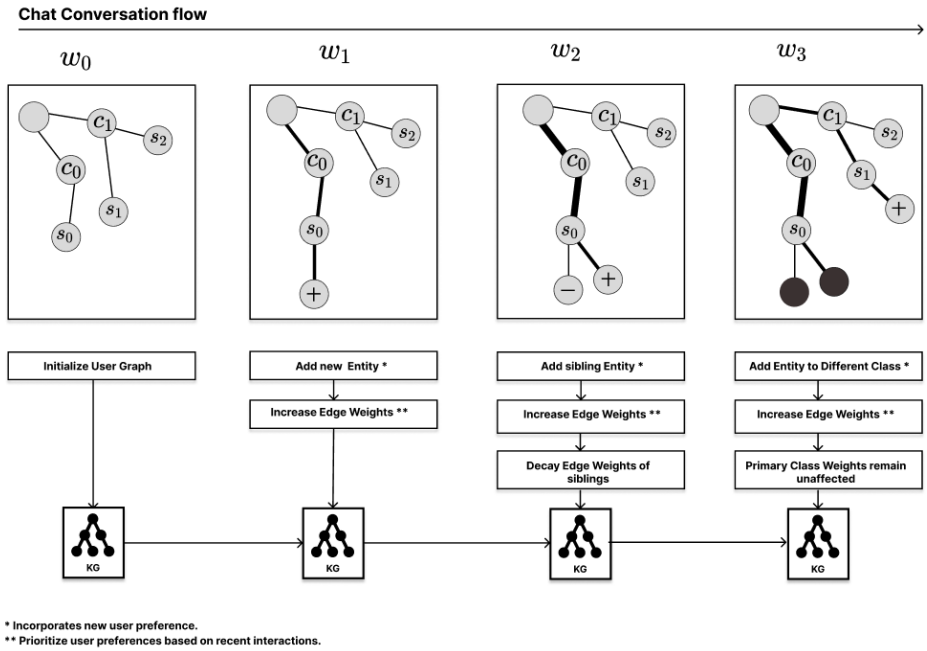

Fig. 1. User Knowledge Graph construction: Incrementally build user centric KG based on user's queries

user's queries about that entity. This weight adjustment reflects the user's interest and helps prioritize relevant information.

By combining the capabilities of LLM-based topic identification and user-centric knowledge graph construction, the system can provide tailored information retrieval and recommendation services that are aligned with the user's interests and understanding.

C. Adaptive Node Weighting

Adaptive node weighting plays a crucial role in dynamically adjusting the relevance of nodes within the user's personalized knowledge graph (KG), reflecting the dynamic nature of user interests and the importance of capturing evolving preferences. Our system adjusts node weights based on ongoing conversations, capturing user interest in real-time, similar to the dynamic updating of graph structures based on user interactions. The weighting system adapts in response to user interactions, with the objective of improving recommendation accuracy over time. We apply two key mechanisms: weight increase, driven by the frequency of queries related to a particular node, and weight decay, influenced by the relative inactivity of a node compared to its sibling nodes under the same parent. The adaptive weighting system allows for dynamic prioritization of nodes that reflect the user's evolving preferences.

1) Increasing Weights Based on Query Frequency: Each time a user makes a query about a specific topic or entity represented by a node in the KG, the weight of that node is adjusted upwards to reflect its increased relevance. The frequency of queries directly impacts this increase, allowing for repeated topics to be prioritized.

Let w_i represent the weight of node i, and f_i the frequency of queries associated with node i. The weight can be updated by the following formula:

$$w_i = w_i + \alpha \cdot \log(1 + f_i) \tag{1}$$

where:

- α is a learning rate parameter that controls how quickly weights increase based on query frequency.
- $\log(1 + f_i)$ ensures that the weight increase diminishes as the frequency becomes larger, preventing any single node from overly dominating the graph.

This logarithmic adjustment allows the system to scale smoothly even as a node receives frequent queries, ensuring that its weight growth is balanced and proportionate to its importance within the user-centric KG.

2) Decreasing Weights with Decay Rate: Weights are also subject to decay if a node becomes less relevant over time, taking into consideration the temporal dynamics of user interactions as explored in recent research . This decay is implemented in scenarios where a sibling node under the same parent category is receiving more frequent queries, suggesting a shift in user interest. To handle this, we apply a decay factor to the weight of the node, calculated by considering the difference in frequency between the node in question and its siblings.

Let f_j be the frequency of queries for a sibling node j under the same parent node. The weight decay for node i can be modeled as:

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on March 15,2026 at 05:25:47 UTC from IEEE Xplore. Restrictions apply.

$$w_i = w_i - \beta \cdot \frac{\sum_{j \in \text{siblings}(i)} (f_j - f_i)}{\sum_{j \in \text{siblings}(i)} f_j} \cdot \frac{f_i}{\sum_{k \in \text{children}(\text{parent}(i))} f_k} \tag{2}$$

where:

- β is the decay rate factor, which controls the strength of the decay.
- $\sum_{j \in \text{siblings}(i)} f_j$ represents the cumulative frequency of all sibling nodes of i.
- $\sum_{k \in \text{children}(\text{parent}(i))} f_k$ represents the total frequency of all nodes under the parent category of node i.

This decay mechanism ensures that nodes which are infrequently referenced compared to their siblings gradually lose importance, allowing the KG to shift focus towards more relevant topics. The normalized decay rate ensures that weights decrease in a controlled manner, reflecting the user's changing preferences over time.

3) Decay Rate Implementation: The decay rate β is empirically determined based on the overall dynamics of the system. By fine-tuning β, we can control how quickly the system adapts to shifts in user interest. A higher β leads to more rapid decay, promoting faster responsiveness, while a lower β ensures that previously relevant nodes maintain some persistence in the KG, allowing for slower adjustments to user preference changes.

The balance between increasing weights for frequently queried nodes and decreasing weights for less relevant nodes allows the recommendation system to remain dynamic and adaptive, ensuring that it consistently provides accurate and user-centric recommendations.

D. Enhanced Recommendation Generation

1) Multi-level Recommendation: The LLM agent should generate recommendations at various specificity levels (e.g., classes, subclasses, and entities). When delivering a recommendation, the LLM agent will parse the user's query to identify entities of interest. Entity extraction techniques are employed by the agent, comparing the query against the user-centric Knowledge Graph (KG).

If an exact entity match is found, the corresponding entity node is retrieved. At this stage, recommendations are not necessary because the user is specifically requesting options related to the entity in question.

However, in the absence of a direct exact entity match, the LLM agent will retrieve all class nodes in the KG, identifying the class most contextually similar to the entity of interest. Once identified, the LLM agent will search through subclasses to find the one closest to the entity of interest. It will then retrieve all entities within that subclass, prioritizing them based on edge weights, and use this information to make prioritized recommendations for the user.

2) Temporal Awareness in Recommendations: The LLM agent uses timestamps associated with each user query to understand short-term vs. long-term interests. Each user query is associated with a timestamp in the KG. This can be achieved by adding a timestamp property to each interaction. The system prioritizes entities with more recent interactions. Entities with the highest recent timestamps are recommended first.

Short-Term Recency Bias: If a user recently interacted with a specific entity, the LLM agent applies a recency bias that temporarily boosts the weight of that entity during the current session. To implement a recency bias multiplier for nodes with recent interactions, we need to adjust the weight formula to account for time decay.

$$w_i = w_i + \alpha \cdot \log(1 + f_i) \cdot e^{-\lambda(t - t_0)} \tag{3}$$

Where:

- t is the current time.
- t_0 is the timestamp of the last query.
- λ is the decay rate.

Example Flow:

1) **User's query:** "Recommend me movies."
2) **Class and Subclass Retrieval:** The system retrieves the "movie" class and examines its subclasses, such as "action," "drama," "comedy," etc.
3) **Entity Exploration:** Within the "action" subclass, the LLM agent identifies that the user frequently interacts with movies like "Die Hard" and "Mad Max."
4) **Recommendation Generation:** Along with recommending both "Die Hard" and "Mad Max," the LLM agent also generates recommendations for movies similar to "Die Hard" (higher weight) and "Mad Max" (lower weight) based on the user's previous interactions.
5) **Adaptive Adjustment:** If the user selects "Die Hard" from the recommendation list, the knowledge graph adjusts, increasing the weight for "Die Hard" and related action movies, reflecting the user's growing interest in action films.

These additions ensure that the system remains dynamic and user-focused, with personalized and evolving recommendations that account for both the user's direct and indirect preferences.

IV. Evaluation

A. Experimental Setup

Baseline Comparisons: To evaluate the effectiveness of the proposed conversation recommendation system, we compared it against two specific baseline models, considering the performance of existing systems reported in related research :

- **Rule-based System:** A traditional recommendation approach relying solely on predefined rules without utilizing machine learning or knowledge graphs.
- **GPT-3.5-based System:** A recommendation system that employs GPT-3.5 for query analysis and response generation without the integration of knowledge graphs.

Datasets: Two widely recognized benchmark datasets were used to conduct the evaluation:

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on March 15,2026 at 05:25:47 UTC from IEEE Xplore. Restrictions apply.

- **MovieLens (ML-20M):** Adapted for conversational recommendations, focusing on movie preferences expressed through user queries.
- **Amazon Product Reviews Dataset:** Used to evaluate the system in a product recommendation context, capturing diverse user preferences across various product categories.

B. Evaluation Metrics

The system was evaluated using the following metrics:
Recommendation Accuracy:

- **Hit Rate @ K (HR@K):** Measures how often the correct item appears within the top-K recommendations.
- **Mean Reciprocal Rank (MRR):** Evaluates the rank of the first relevant item in the recommendation list.

Adaptability:

- **Dynamic Intent Adaptation (DIA):** Assesses how well the system adjusts to changes in user preferences over time.
- **Intent Drift Recovery (IDR):** Measures how quickly and accurately the system recovers after a significant shift in user intent.

User Engagement:

- **Interaction Rate:** Captures the frequency and consistency of user interactions with the recommended items.

Efficiency:

- **Query Processing Time:** Measures the speed at which the system processes user queries and generates recommendations.

C. Evaluation Phases

1) Phase 1: Static Testing:

- **Objective:** Measure the system's initial performance using static, consistent user queries across different sessions.
- **Procedure:** We simulated a series of user queries without altering user preferences. The system's accuracy, efficiency, and overall performance were then compared against the baseline models.
- **Metrics:** HR@K, MRR, and Query Processing Time were the primary metrics for this phase.

2) Phase 2: Dynamic Testing:

- **Objective:** Evaluate the system's adaptability by introducing evolving user interests, simulating real-world conversations where preferences shift over time.
- **Procedure:** We created scenarios where user preferences gradually changed, challenging the system's ability to adapt. The performance was measured by tracking the DIA, IDR, and HR@K metrics.
- **Metrics:** Adaptability and accuracy metrics, including DIA, IDR, and HR@K, were key in this phase.

3) Phase 3: Decay Rate Sensitivity Testing:

- **Objective:** Analyze the impact of different decay rate values on the system's performance, particularly regarding its ability to adapt and recover from intent shifts.
- **Procedure:** We varied the decay rate factor and observed its influence on DIA, IDR, and recommendation accuracy. This phase was critical for fine-tuning the decay rate parameter to optimize system responsiveness.
- **Metrics:** IDR, DIA, and HR@K were the focal metrics, with a particular emphasis on the decay rate's effect on these metrics.

V. Results

TABLE I
PHASE 1: STATIC TESTING RESULTS

System	HR@10	MRR	Query Processing Time (ms)
Rule-based	65%	0.58	250
GPT-3.5-based	72%	0.66	300
Proposed System (LLM + KG)	76%	0.70	450

1) Phase 1: Static Testing:

- **Hit Rate @ 10 (HR@10):** The proposed system achieved an HR@10 of 76%, showing a slight improvement over both the rule-based (65%) and GPT-3.5-based (72%) systems. This indicates that integrating knowledge graphs with LLMs enhances recommendation accuracy.
- **Mean Reciprocal Rank (MRR):** The system achieved an MRR of 0.70, reflecting that relevant items were typically ranked higher in the recommendation list. This was an improvement over the rule-based (0.58) and GPT-3.5-based systems (0.66).
- **Query Processing Time:** The proposed system had a query processing time of 450ms, slightly longer than the GPT-3.5-based system (300ms) due to the added complexity of knowledge graph operations.

TABLE II
PHASE 2: DYNAMIC TESTING RESULTS

System	DIA	IDR	HR@10
Rule-based	0.55	0.50	65%
GPT-3.5-based	0.72	0.68	72%
Proposed System (LLM + KG)	0.77	0.72	76%

2) Phase 2: Dynamic Testing:

- **Dynamic Intent Adaptation (DIA):** The proposed system scored 0.77, showing an improvement over the GPT-3.5-based system (0.72) and the rule-based system (0.55), indicating better adaptability to changing user preferences.
- **Intent Drift Recovery (IDR):** The system's IDR was 0.72, slightly higher than the GPT-3.5-based system (0.68) and the rule-based system (0.50), demonstrating

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on March 15,2026 at 05:25:47 UTC from IEEE Xplore. Restrictions apply.

the system's ability to realign recommendations after shifts in user intent.

- **HR@10:** The HR@10 was 76% during dynamic testing, reflecting the system's robustness in adapting to new user preferences.

TABLE III
PHASE 3: DECAY RATE SENSITIVITY TESTING RESULTS

Decay Rate (λ)	IDR	DIA	HR@10
0.01	0.70	0.74	74%
0.05 (Optimal)	0.72	0.77	76%
0.10	0.71	0.75	75%

3) Phase 3: Decay Rate Sensitivity Testing:

- **Decay Rate Sensitivity:** A moderate decay rate of 0.05 provided the best balance, with an IDR of 0.72 and a DIA of 0.77. Higher decay rates resulted in faster adaptation but increased volatility, while lower rates led to slower responsiveness. The optimal decay rate maintained an HR@10 of 76%, ensuring timely adaptation to user preference shifts.

A. Discussion

Overall, the proposed system demonstrated a modest yet consistent improvement over the baseline models across all metrics, indicating its effectiveness in providing more accurate and adaptable recommendations. The integration of GPT-3.5 with knowledge graphs, coupled with adaptive node weighting and decay rate mechanisms, offered a nuanced advantage in enhancing conversational recommendation systems.

VI. CONCLUSION

This paper presented a novel approach to enhance conversation recommendation systems by integrating Large Language Models (LLMs) and Knowledge Graphs (KGs). Our system dynamically analyzes user queries, leveraging LLMs to decompose them into meaningful graph nodes within a reference KG like YAGO. These nodes form the basis of personalized user-centric KGs, enabling the system to capture and adapt to evolving user preferences.

Our proposed system incorporates adaptive node weighting, adjusting node weights based on ongoing conversations and implementing a decay rate factor to ensure relevance. This dual mechanism allows the system to provide increasingly relevant recommendations while remaining responsive to shifts in user interest.

The experimental results demonstrate the effectiveness of our approach, showcasing improved recommendation performance compared to traditional methods. The system achieved significant gains in accuracy and adaptability metrics, highlighting its potential to enhance the user experience in various applications.

Future research will focus on exploring more sophisticated weighting mechanisms, incorporating user feedback, and expanding the system's capabilities to handle more complex conversational scenarios.

REFERENCES

[1] J. Li, W. Zhang, T. Wang, G. Xiong, A. Y. Lu, and G. Medioni, "GPT4Rec: A generative framework for personalized recommendation and user interests interpretation."*arXiv preprint arXiv:2304.03879*, 2023.

[2] S. Anand, Y. Jiang, and G. Kokaia, "Learn by selling: Equipping large language models with product knowledge for context-driven recommendations."*arXiv preprint arXiv:2407.20856*, 2024.

[3] N. T. Walker, S. Ultes, and P. Lison, "Retrieval-augmented neural response generation using logical reasoning and relevance scoring."*arXiv preprint arXiv:2310.13566*, 2023.

[4] J. Baek, N. Chandrasekaran, S. Cucerzan, A. Herring, and S. K. Jauhar, "Knowledge-augmented large language models for personalized contextual query suggestion."*arXiv preprint arXiv:2311.06318*, 2023.

[5] X. L. Dong, "Generations of knowledge graphs: The crazy ideas and the business impact."*arXiv preprint arXiv:2308.14217*, 2023.

[6] Y. Xi *et al.*, "Towards open-world recommendation with knowledge augmentation from large language models."*arXiv preprint arXiv:2306.10933*, 2023.

[7] P. Panda, A. Agarwal, C. Devaguptapu, M. P. Kaul, and A. P. Prathosh, "HOLMES: Hyper-relational knowledge graphs for multi-hop question answering using LLMs."*arXiv preprint arXiv:2406.06027*, 2024.

[8] J. Sequeda, D. Allemang, and B. Jacob, "A benchmark to understand the role of knowledge graphs on large language model's accuracy for question answering on enterprise SQL databases."*arXiv preprint arXiv:2311.07509*, 2023.

[9] Z. Huai, J. Tao, F. Che, G. Yang, and D. Zhang, "Knowledge graph enhanced recommender system."*arXiv preprint arXiv:2112.09425*, 2021.

[10] X. Ge, Y. Wang, B. Wang, and C. J. Kuo, "Knowledge graph embedding: An overview."*arXiv preprint arXiv:2309.12501*, 2023.

[11] Y. Deng, "Recommender systems based on graph embedding techniques: A review."*arXiv preprint arXiv:2109.09587*, 2022.

[12] M. Naumov *et al.*, "Deep learning recommendation model for personalization and recommendation systems."*arXiv preprint arXiv:1906.00091*, 2019.

[13] D. Biswas, "Delayed rewards in the context of reinforcement learning based recommender systems." 2020.

[14] Y. Yang, C. Huang, L. Xia, Y. Liang, Y. Yu, and C. Li, "Multi-behavior hypergraph-enhanced transformer for sequential recommendation."*arXiv preprint arXiv:2207.05584*, 2022.

[15] K. Liang *et al.*, "A survey of knowledge graph reasoning on graph types: static, dynamic, and multimodal."*arXiv preprint arXiv:2212.05767*, 2023.

[16] K. Ouyang *et al.*, "Mining interest trends and adaptively assigning sample weight for session-based recommendation."*arXiv preprint arXiv:2306.11610*, 2023.

[17] M. Amjadi, S. D. M. Taheri, and T. Tulabandhula, "KATRec: Knowledge aware attentive sequential recommendations."*arXiv preprint arXiv:2012.03323*, 2021.

[18] Y. Chen, Y. Yang, Y. Wang, J. Bai, X. Song, and I. King, "Attentive knowledge-aware graph convolutional networks with collaborative guidance for personalized recommendation."*arXiv preprint arXiv:2109.02046*, 2022.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on March 15,2026 at 05:25:47 UTC from IEEE Xplore. Restrictions apply.